[Euron] 머신러닝 스터디 9주차

조윤서 · 방금 전

0

(Euron) 머신러닝

목록 보기 6/6

교재: 파이썬 머신러닝 완벽 가이드 (권철민)

제 5장. 회귀

09. 회귀 실습 - 자전거 대여 수요 예측

- datetime: hourly date + timestamp
- season: 1 = 봄, 2 = 여름, 3 = 가을, 4 = 겨울
- holiday: 1 = 토, 일요일의 주말을 제외한 국경일 등의 휴일, 0 = 휴일이 아닌 날
- workingday: 1 = 토, 일요일의 주말 및 휴일이 아닌 주중, 0 = 주말 및 휴일
- weather:
  - 1 = 맑음, 약간 구름 낀 흐림
  - 2 = 안개, 안개 + 흐림
  - 3 = 가벼운 눈, 가벼운 비 + 천둥
  - 4 = 심한 눈/비, 천둥/번개
- temp: 온도(섭씨)
- atemp: 체감온도(섭씨)
- humidity: 상대습도
- windspeed: 풍속
- casual: 사전에 등록되지 않는 사용자가 대여한 횟수
- registered: 사전에 등록된 사용자가 대여한 횟수
- count: 대여 횟수

- 결정 값: `count` 칼럼으로 '대여 횟수'를 의미

1. 데이터 클렌징 및 가공과 데이터 시각화

(1) 데이터 확인

```
bike_df=pd.read_csv('bike_train.csv')
print(bike_df.shape)
bike_df.head()
```

```
(10886, 12)
```

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	0.0	0	1	1

```
bike_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   datetime    10886 non-null  object
 1   season      10886 non-null  int64
 2   holiday     10886 non-null  int64
 3   workingday  10886 non-null  int64
 4   weather     10886 non-null  int64
 5   temp        10886 non-null  float64
 6   atemp       10886 non-null  float64
 7   humidity    10886 non-null  int64
 8   windspeed   10886 non-null  float64
 9   casual      10886 non-null  int64
 10  registered  10886 non-null  int64
 11  count       10886 non-null  int64
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
```

- 10886개의 레코드와 12개의 칼럼으로 구성
- Null 없음
- datetime 칼럼만 object형 -> 가공 필요

(2) 데이터 전처리1 - 칼럼 수정

(2)-1. datetime

```
bike_df['datetime']=bike_df.datetime.apply(pd.to_datetime)

bike_df['year'] = bike_df.datetime.apply(lambda x:x.year)
bike_df['month'] = bike_df.datetime.apply(lambda x:x.month)
bike_df['day'] = bike_df.datetime.apply(lambda x:x.day)
bike_df['hour'] = bike_df.datetime.apply(lambda x:x.hour)
bike_df.head(3)
```

y	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count	year	month	day	hour
0	0	1	9.84	14.395	81	0.0	3	13	16	2011	1	1	0
0	0	1	9.02	13.635	80	0.0	8	32	40	2011	1	1	1
0	0	1	9.02	13.635	80	0.0	5	27	32	2011	1	1	2

- 문자열을 datetime 타입으로 변경
- 년, 월, 일, 시간 칼럼 추출

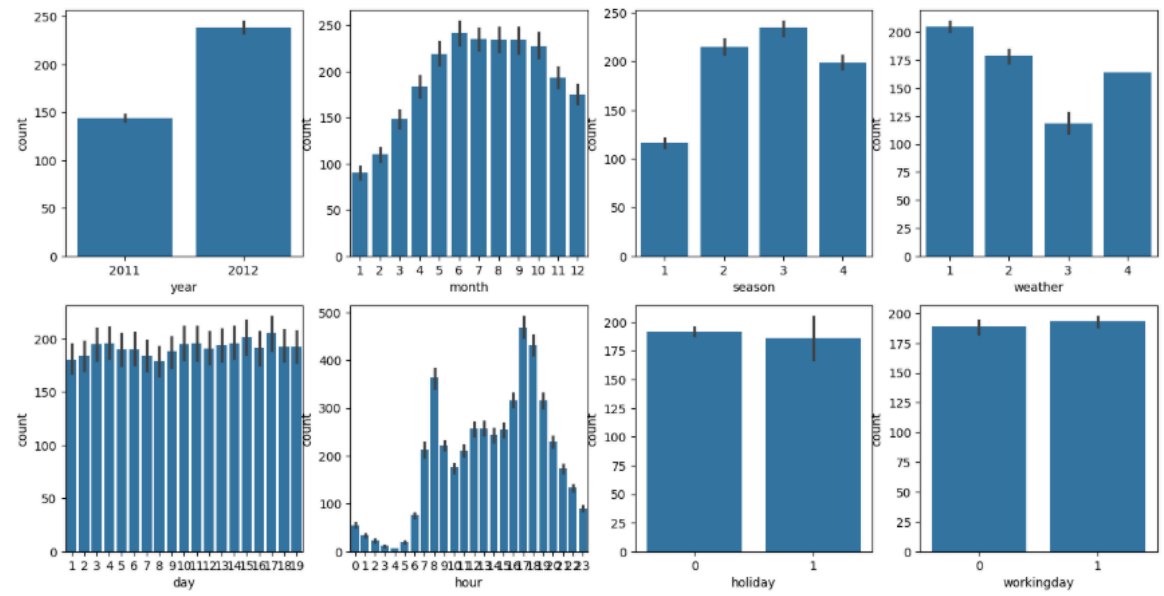
```
drop_columns=['datetime', 'casual', 'registered']
bike_df.drop(drop_columns, axis=1, inplace=True)
```

- `datetime` 칼럼 삭제

- casual + registered = count 이므로 `casual` 과 `registerd` 칼럼 삭제

(3) 데이터 전처리2 - 시각화

```
fig, axs = plt.subplots(figsize=(16, 8), ncols=4, nrows=2)
cat_features=['year', 'month', 'season', 'weather', 'day', 'hour', 'holiday', 'workingday']
for i, feature in enumerate(cat_features):
  row = int(i/4)
  col = i%4
  sns.barplot(x=feature, y='count', data=bike_df, ax=axs[row][col])
```



- 타겟 값인 `count` 분포 시각화
- `year` : 2012년이 2011년보다 상대적으로 값이 높음
- `month` : 1, 2, 3월이 낮고 6, 7, 8, 9월이 높음
- `season` : 봄(1), 겨울(4)이 낮고 여름(2), 가을(3)이 높음
- `weather` : 눈 또는 비가 있는 경우(3, 4)가 낮고, 맑거나(1) 약간 안개가 있는 경우(2)가 높음
- `hour` : 오전 출근 시간(8)과 오후 퇴근 시간(17, 18)이 상대적으로 높음
- `day` : 일자간의 차이는 크지 않음
- `holiday` , `workingday` : 휴일보다는 주중일 경우가 상대적으로 약간 높음

(4) RMSLE, MAE, RMSE 평가 함수 만들기

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

def rmsle(y, pred):
  log_y = np.log1p(y)
  log_pred = np.log1p(pred)
  squared_error = (log_y - log_pred) ** 2
  rmsle = np.sqrt(np.mean(squared_error))
  return rmsle

def rmse(y, pred):
  return np.sqrt(mean_squared_error(y, pred))

def evaluate_regr(y, pred):
  rmsle_val = rmsle(y, pred)
  rmse_val = rmse(y, pred)
  mae_val = mean_absolute_error(y, pred)
  print('RMSLE: {:.3f}, RMSE: {:.3f}, MAE: {:.3f}'.format(rmsle_val, rmse_val, mae_val))
```

- `rmsle` 을 구할 때 넘파이의 `log()` 함수를 이용하거나 사이킷런의 `mean_squared_log_error()` 를 이용할 수 있지만 데이터 값의 크기에 따라 오버플로/언더플로 오류가 발생가히 쉬움 -> `log1p()` 를 이용하는 것이 좋음
- `logp1(x)` 의 경우는 log(1+x)로 변환되므로 x 값이 0이어도 무한대가 되지 않아 오버플로/언더플로 문제를 해결 + 넘파이의 `expm1()` 함수로 쉽게 원래의 스케일로 복원됨

로그 변환, 피처 인코딩과 모델 학습/예측/평가

(1) 회귀 예측

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso

y_target = bike_df['count']
X_features = bike_df.drop(['count'], axis=1, inplace=False)

X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.3, random_state=0)

lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
pred = lr_reg.predict(X_test)

evaluate_regr(y_test, pred)
```

```
RMSLE: 1.165, RMSE: 140.900, MAE: 105.924
```

```
def get_top_error_data(y_test, pred, n_tops=5):
  result_df = pd.DataFrame(y_test.values, columns=['real_count'])
  result_df['predicted_count']=np.round(pred)
  result_df['diff'] = np.abs(result_df['real_count'] - result_df['predicted_count'])

  print(result_df.sort_values('diff', ascending=False)[:n_tops])

get_top_error_data(y_test, pred, n_tops=5)
```

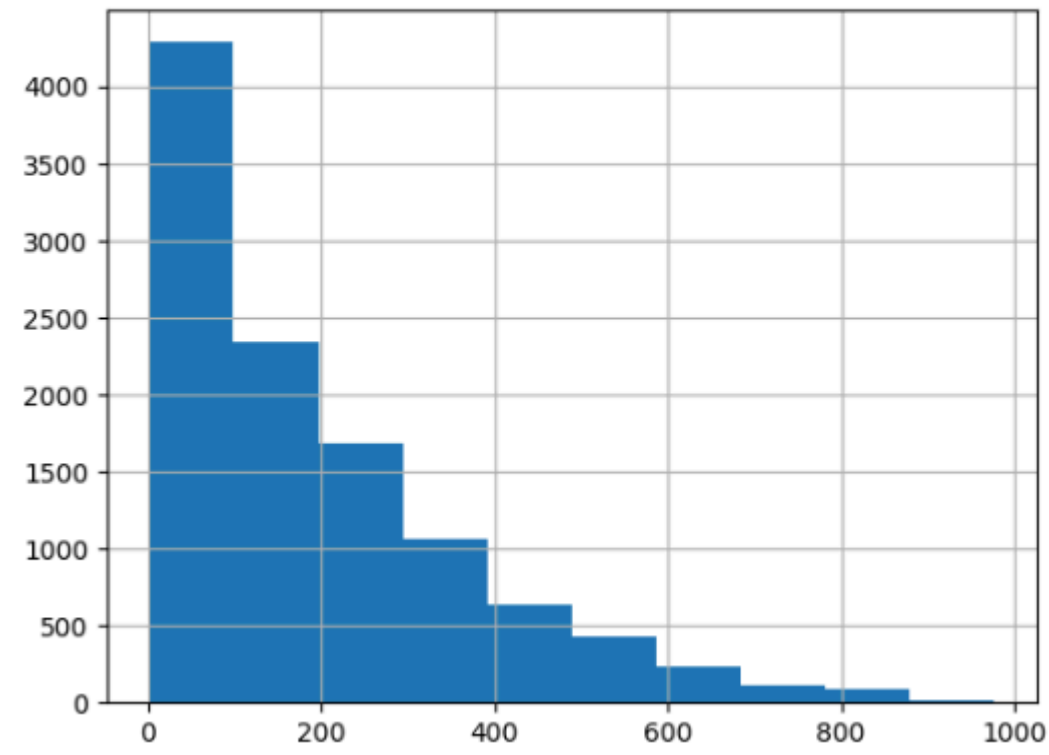
```
      real_count  predicted_count   diff
1618         890            322.0  568.0
966          884            327.0  557.0
3151         798            241.0  557.0
412          745            194.0  551.0
2817         856            310.0  546.0
```

- RMSLE: 1.165, RMSE: 140.900, MAE: 105.924
- 가장 큰 상위 5위 오류 값은 546~568
- 실제 값을 감안하면 예측 오류가 꽤 큼 -> target 값의 분포 확인

(1)-2. target 값 분포 확인

```
y_target.hist()
```

```
<Axes: >
```

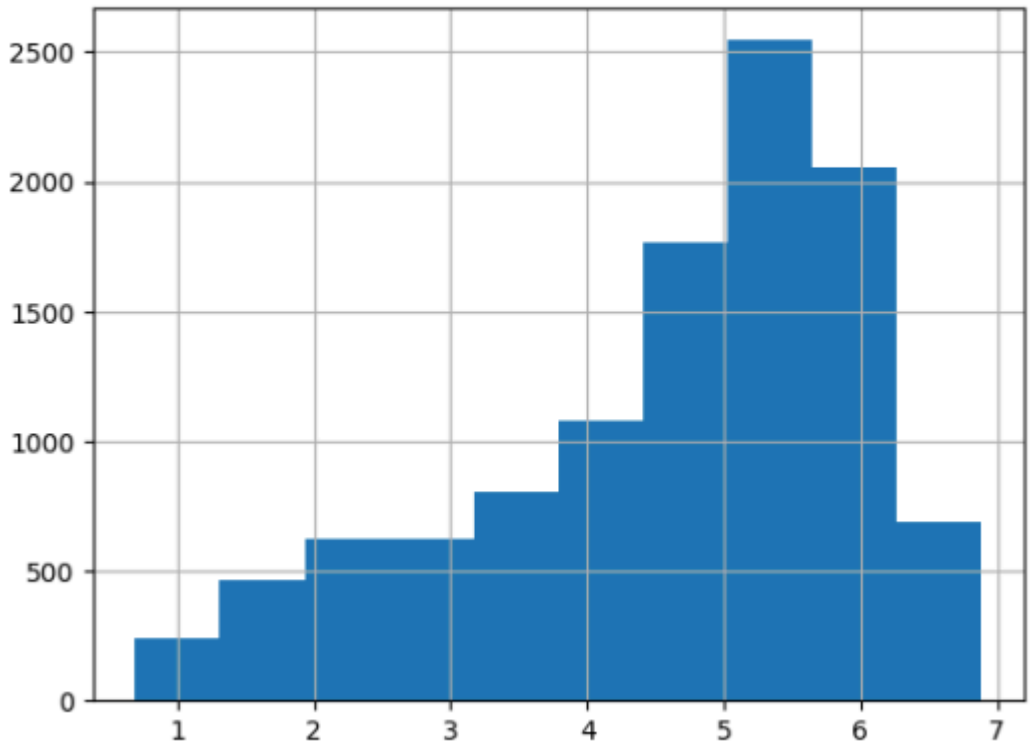


- `count` 칼럼 값이 정규 분포가 아님
- 0~200 사이에 왜곡되어 있음

(1)-3. 로그 변환

```
y_log_transform=np.log1p(y_target)
y_log_transform.hist()
```

```
<Axes: >
```



- target 값을 로그로 변환 -> 왜곡 정도가 많이 개선

(1)-4. 재학습 및 평가 수행

```
y_target_log = np.log1p(y_target)

X_train, X_test, y_train, y_test = train_test_split(X_features, y_target_log,
                                                    test_size=0.3,random_state=0)

lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
pred = lr_reg.predict(X_test)

y_test_exp = np.expm1(y_test)
pred_exp=np.expm1(pred)

evaluate_regr(y_test_exp, pred_exp)
```

```
RMSLE: 1.017, RMSE: 162.594, MAE: 109.286
```

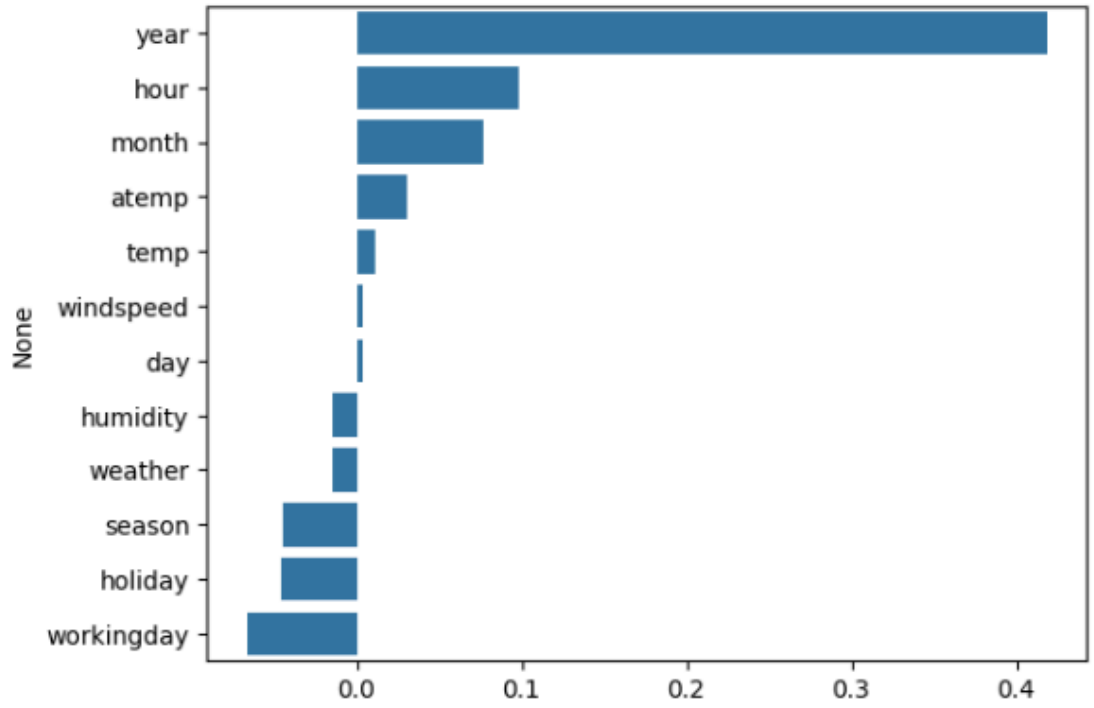
- 로그로 변환된 target 값을 기반으로 학습 후 예측한 값은 `expml()` 함수를 적용해 원래 scale 값으로 원상 복구
- RMSLE 오류는 줄어들었지만 RMSE는 오히려 늘어남

(2) 개별 피처들 인코딩 적용

(2)-1. 각 피처의 회귀 계숫값 시각화

```
coef = pd.Series(lr_reg.coef_, index=X_features.columns)
coef_sort = coef.sort_values(ascending=False)
sns.barplot(x=coef_sort.values, y=coef_sort.index)
```

```
<Axes: ylabel='None'>
```



- `year` , `hour` , `month` , `season` , `holiday` , `workingday` 피처들의 회귀 계수 영향도가 상대적으로 높음
- `year` , `hour` , `month` 등은 숫자 값으로 표현되어있지만 모두 카테고리 형 피처 -> 원-핫 인코딩 필요

(2)-2. 원-핫 인코딩 및 예측 성능 확인

```
X_features_ohe = pd.get_dummies(X_features, columns=['year', 'month', 'day', 'hour',
                                                     'holiday', 'workingday', 'season', 'weather'])
```

```
X_train, X_test, y_train, y_test = train_test_split(X_features_ohe, y_target_log,
                                                    test_size=0.3, random_state=0)

def get_model_predict(model, X_train, X_test, y_train, y_test, is_expm1=False):
  model.fit(X_train, y_train)
  pred = model.predict(X_test)
  if is_expm1:
    y_test = np.expm1(y_test)
    pred = np.expm1(pred)
  print('###', model.__class__.__name__, '###')
  evaluate_regr(y_test, pred)

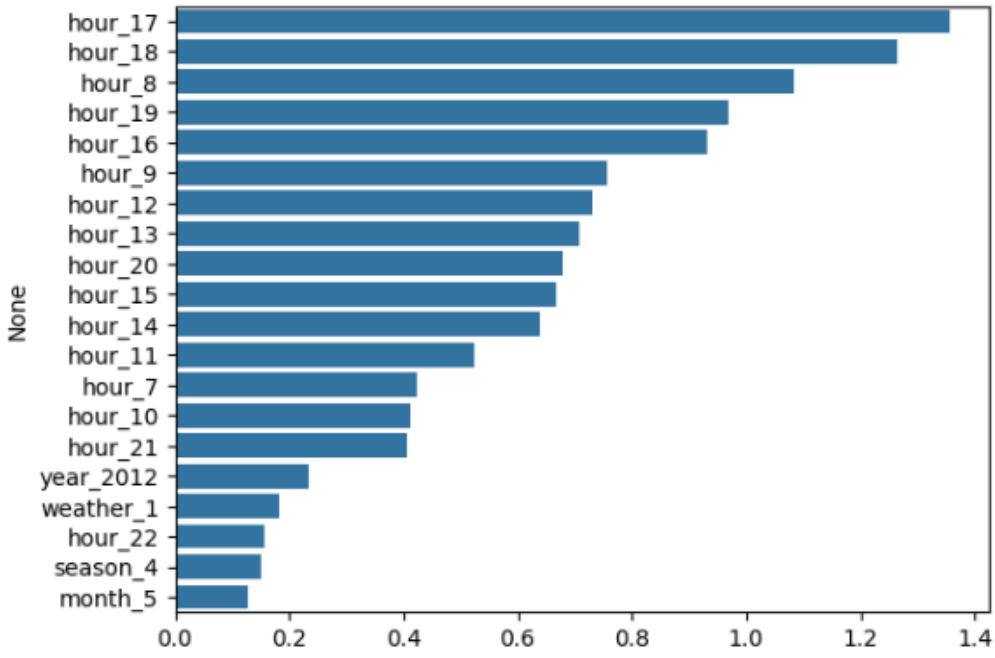
lr_reg = LinearRegression()
ridge_reg = Ridge(alpha=10)
lasso_reg = Lasso(alpha=0.01)

for model in [lr_reg, ridge_reg, lasso_reg]:
  get_model_predict(model, X_train, X_test, y_train, y_test, is_expm1=True)
```

```
### LinearRegression ###
RMSLE: 0.590, RMSE: 97.688, MAE: 63.382
### Ridge ###
RMSLE: 0.590, RMSE: 98.529, MAE: 63.893
### Lasso ###
RMSLE: 0.635, RMSE: 113.219, MAE: 72.803
```

```
coef = pd.Series(lr_reg.coef_, index=X_features_ohe.columns)
coef_sort = coef.sort_values(ascending=False)[:20]
sns.barplot(x=coef_sort.values, y=coef_sort.index)
```

```
<Axes: ylabel='None'>
```



- 인코딩 후, 예측 성능이 많이 향상됨
- 여전히 `year` 관련 피처들의 회귀 계수 값이 가장 높지만, `season`, `weather` 과 관련된 속성들의 회기 계수 값도 상대적으로 커짐

(3) 회귀 예측(with 회귀 트리)

```
import warnings
warnings.filterwarnings('ignore')
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

rf_reg = RandomForestRegressor(n_estimators=500)
gbm_reg = GradientBoostingRegressor(n_estimators=500)
xgb_reg = XGBRegressor(n_estimators=500)
lgbm_reg = LGBMRegressor(n_estimators=500, verbose=-1)

for model in [rf_reg, gbm_reg, xgb_reg, lgbm_reg]:
  get_model_predict(model, X_train.values, X_test.values, y_train.values,
                    y_test.values, is_expm1=True)
```

```
### RandomForestRegressor ###
RMSLE: 0.355, RMSE: 50.324, MAE: 31.116
### GradientBoostingRegressor ###
RMSLE: 0.330, RMSE: 53.346, MAE: 32.748
### XGBRegressor ###
RMSLE: 0.339, RMSE: 51.475, MAE: 31.357
### LGBMRegressor ###
RMSLE: 0.319, RMSE: 47.215, MAE: 29.029
```

- 앞의 선형 회귀 모델보다 예측 성능이 개선됨

10. 회귀 실습 - 캐글 주택 가격: 고급 회귀 기법

- 미국 아이오와 주의 에임스 지방의 주택 가격 정보 데이터셋

데이터 사전 처리

(1) 데이터 살펴보기

```
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

house_df_org = pd.read_csv('house_price.csv')
house_df = house_df_org.copy()
house_df.head(3)
```

ies	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	SalePrice
Pub	...	0	NaN	NaN	NaN	0	2	2008	WD	Normal	208500
Pub	...	0	NaN	NaN	NaN	0	5	2007	WD	Normal	181500
Pub	...	0	NaN	NaN	NaN	0	9	2008	WD	Normal	223500

- target 값은 `SalePrice` 칼럼

```
print('데이터 세트의 Shape:', house_df.shape)
print('₩n전체 피처의 type ₩n', house_df.dtypes.value_counts())
isnull_series = house_df.isnull().sum()
print('₩nNull 칼럼과 그 건수:₩n',
      isnull_series[isnull_series >0].sort_values(ascending=False))
```

```
데이터 세트의 Shape: (1460, 81)

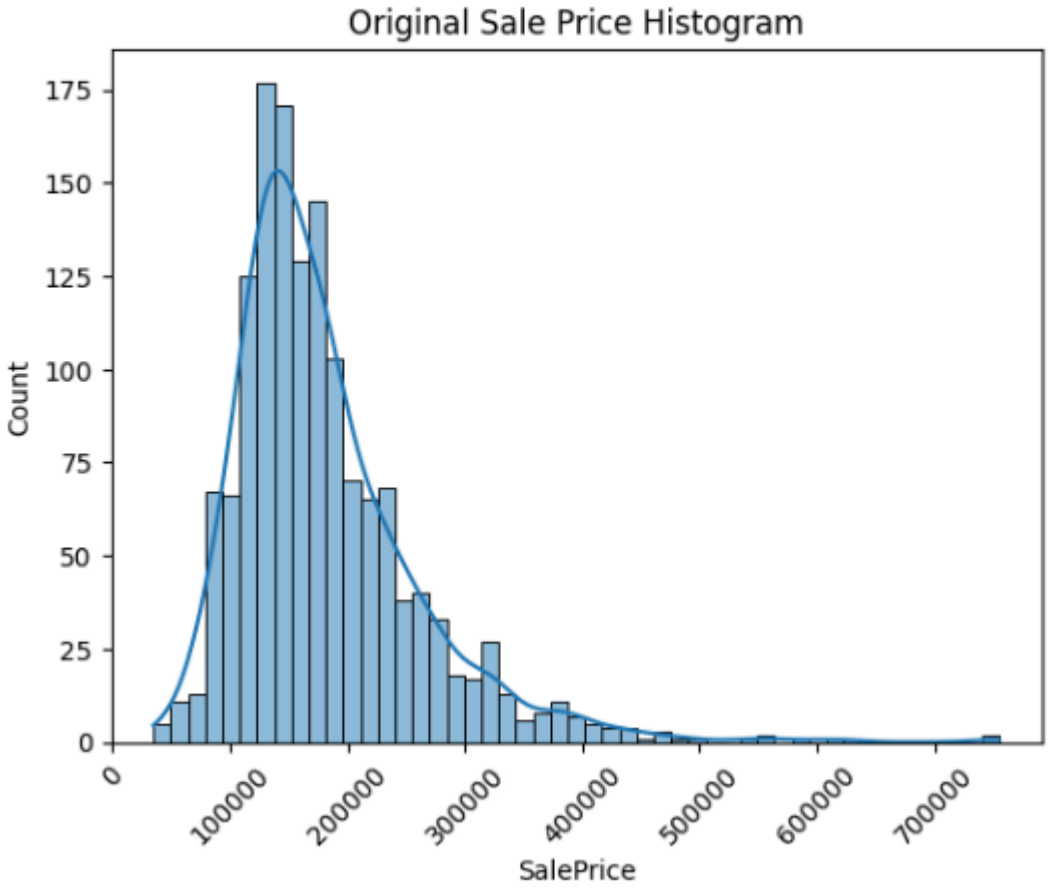
전체 피처의 type
 object     43
int64      35
float64     3
Name: count, dtype: int64

Null 칼럼과 그 건수:
 PoolQC          1453
MiscFeature     1406
Alley           1369
Fence           1179
MasVnrType       872
FireplaceQu      690
LotFrontage      259
GarageType        81
GarageYrBlt       81
GarageFinish      81
GarageQual        81
GarageCond        81
BsmtExposure      38
BsmtFinType2      38
BsmtQual          37
BsmtCond          37
BsmtFinType1      37
MasVnrArea         8
Electrical         1
dtype: int64
```

- 1460개의 레코드와 81개의 피처로 구성
- target을 제외한 80개의 피처 중 43개가 문자형, 나머지가 숫자형
- `PoolQC` , `MiscFeature` , `Alley` , `Frence` 는 1000개가 넘는 데이터가 Null -> 드롭

(1)-2. 분포 시각화

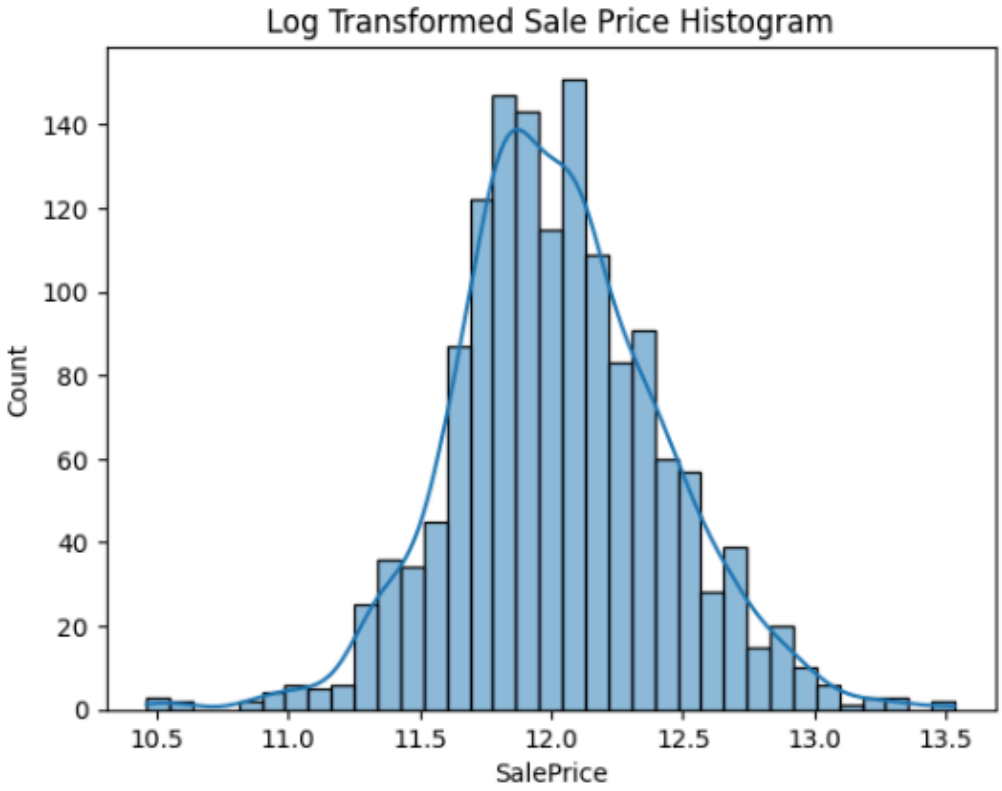
```
plt.title('Original Sale Price Histogram')
plt.xticks(rotation=45)
sns.histplot(house_df['SalePrice'], kde=True)
plt.show()
```



- 왼쪽으로 치우친 형태로, 정규 분포에서 벗어나 있음 -> 로그 적용 필요

(1)-2. 로그 적용

```
plt.title('Log Transformed Sale Price Histogram')
log_SalePrice = np.log1p(house_df['SalePrice'])
sns.histplot(log_SalePrice, kde=True)
plt.show()
```




```
original_SalePrice = house_df['SalePrice']
house_df['SalePrice'] = np.log1p(house_df['SalePrice'])
```

- 로그 변환 후 정규 분포 형태로 결괏값이 분포
- `SalePrice` 를 로그 변환한 뒤 DataFrame에 반영

(2) 칼럼 수정 및 삭제

```
house_df.drop(['Id', 'PoolQC', 'MiscFeature', 'Alley', 'Fence', 'FireplaceQu'],
              axis=1, inplace=True)

house_df.fillna(house_df.mean(numeric_only=True), inplace=True)

null_column_count = house_df.isnull().sum()[house_df.isnull().sum()>0]
print('## Null 피처의 Type: ₩n', house_df.dtypes[null_column_count.index])
```

```
## Null 피처의 Type:
 MasVnrType      object
BsmtQual         object
BsmtCond         object
BsmtExposure     object
BsmtFinType1     object
BsmtFinType2     object
Electrical       object
GarageType       object
GarageFinish     object
GarageQual       object
GarageCond       object
dtype: object
```

- Null이 많은 칼럼과 불필요한 칼럼은 삭제
- 드롭하지 않은 숫자형 Null 칼럼은 평균값으로 대체

(3) 원-핫 인코딩

```
print('get_dummies() 수행 전 데이터 Shape:', house_df.shape)
house_df_ohe = pd.get_dummies(house_df)
print('get_dummies() 수행 후 데이터 Shape:', house_df_ohe.shape)

null_column_count = house_df_ohe.isnull().sum()[house_df_ohe.isnull().sum() > 0]
print('## Null 피처의 Type: ₩n', house_df_ohe.dtypes[null_column_count.index])
```

```
get_dummies() 수행 전 데이터 Shape: (1460, 75)
get_dummies() 수행 후 데이터 Shape: (1460, 270)
## Null 피처의 Type:
 Series([], dtype: object)
```

- 피처가 75개에서 271개로 증가
- Null 값 존재X

선형 회귀 모델 학습/예측/평가

(1) 모델 학습, 예측, 평가

```
def get_rmse(model):
  pred = model.predict(X_test)
  mse = mean_squared_error(y_test, pred)
  rmse = np.sqrt(mse)
  print(model.__class__.__name__,'로그 변환된 RMSE:', np.round(rmse, 3))
  return rmse

def get_rmses(models):
  rmses=[]
  for model in models:
    rmse = get_rmse(model)
    rmses.append(rmse)
  return rmses
```

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split

y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice', axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.2,
                                                    random_state=156)

lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
ridge_reg = Ridge()
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso()
lasso_reg.fit(X_train, y_train)

models = [lr_reg, ridge_reg, lasso_reg]
get_rmses(models)
```

```
LinearRegression 로그 변환된 RMSE: 0.132
Ridge 로그 변환된 RMSE: 0.127
Lasso 로그 변환된 RMSE: 0.176
```

- 라쏘 회귀의 경우 회귀 성능이 타 회귀 방식보다 많이 떨어짐 -> 파라미터 튜닝 필요

(2) 하이퍼 파라미터 최적화

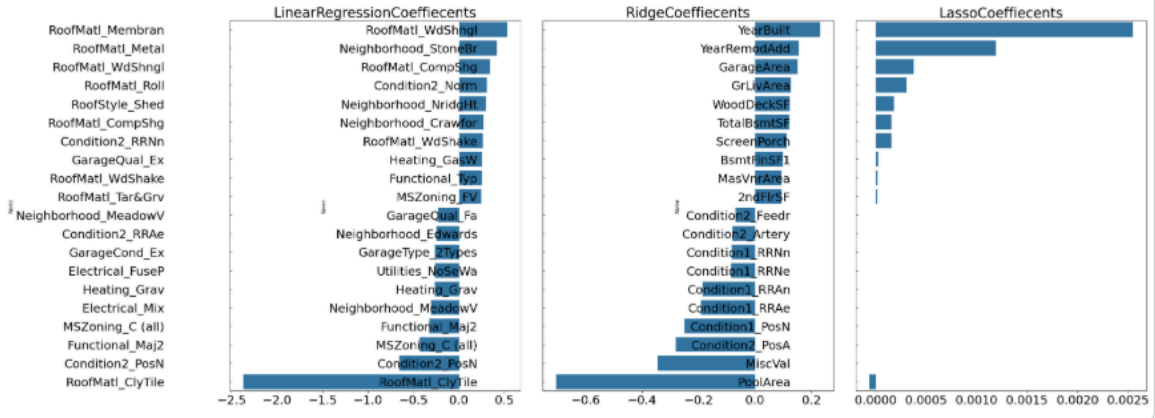
(2)-1. 회귀 계수 시각화

```
def get_top_bottom_coef(model, n=10):
  coef = pd.Series(model.coef_, index=X_features.columns)

  coef_high = coef.sort_values(ascending=False).head(n)
  coef_low = coef.sort_values(ascending=False).tail(n)
  return coef_high, coef_low
```

```
def visualize_coefficient(models):
  fig, axs = plt.subplots(figsize=(24, 10), nrows=1, ncols=3)
  fig.tight_layout()
  for i_num, model in enumerate(models):
    coef_high, coef_low = get_top_bottom_coef(model)
    coef_concat = pd.concat([coef_high, coef_low])
    axs[i_num].set_title(model.__class__.__name__+'Coeffiecents', size=25)
    axs[i_num].tick_params(axis='y', direction='in', pad=120)
    for label in (axs[i_num].get_xticklabels() + axs[i_num].get_yticklabels()):
      label.set_fontsize(22)
    sns.barplot(x=coef_concat.values, y=coef_concat.index, ax=axs[i_num])

models = [lr_reg, ridge_reg, lasso_reg]
visualize_coefficient(models)
```



- OLS 기반의 LineRegression과 Ridge의 경우는 회귀 계수가 유사한 형태로 분포되어 있음
- Lasso는 전체적으로 회귀 계수 값이 매우 작고, 그중 `YearBuilt` 가 가장 크고 다른 피처의 회귀 계수는 너무 작음 -> 데이터 분할 문제 유무 확인

(2)-2. 교차 검증

```
from sklearn.model_selection import cross_val_score

def get_avg_rmse_cv(models):
  for model in models:
    rmse_list = np.sqrt(-cross_val_score(model, X_features, y_target,
                                         scoring='neg_mean_squared_error', cv=5))
    rmse_avg = np.mean(rmse_list)
    print('₩n{} CV RMSE 값 리스트: {}'.format(model.__class__.__name__, np.round(rmse_list, 3)))
    print('{} CV 평균 RMSE 값: {}'.format(model.__class__.__name__, np.round(rmse_avg, 3)))

models = [ridge_reg, lasso_reg]
get_avg_rmse_cv(models)
```

```
Ridge CV RMSE 값 리스트: [0.117 0.154 0.142 0.117 0.189]
Ridge CV 평균 RMSE 값: 0.144

Lasso CV RMSE 값 리스트: [0.161 0.204 0.177 0.181 0.265]
Lasso CV 평균 RMSE 값: 0.198
```

- 여전히 라쏘의 성능이 비교적 떨어짐

(2)-3. 최적의 alpha 값 찾기

```
from sklearn.model_selection import GridSearchCV

def print_best_params(model, params):
  grid_model=GridSearchCV(model, param_grid=params,
                          scoring='neg_mean_squared_error', cv=5)
  grid_model.fit(X_features, y_target)
  rmse = np.sqrt(-1*grid_model.best_score_)
  print('{} 5 CV 시 최적 평균 RMSE 값: {}, 최적 alpha: {}'.format(model.__class__.__name__,
                                                    np.round(rmse, 4), grid_model.best_params_))

ridge_params = {'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20]}
lasso_params = {'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1, 5, 10]}
print_best_params(ridge_reg, ridge_params)
print_best_params(lasso_reg, lasso_params)
```

```
Ridge 5 CV 시 최적 평균 RMSE 값: 0.1418, 최적 alpha: {'alpha': 12}
Lasso 5 CV 시 최적 평균 RMSE 값: 0.142, 최적 alpha: {'alpha': 0.001}
```

- 릿지 모델의 경우 alpha가 12에서 최적 평균 RMSE가 0.1418, 라쏘 모델의 경우 alpha가 0.001에서 최적 평균 RMSE가 0.142
- 라쏘 모델의 경우, 최적화 이후에 예측 성능이 많이 좋아짐

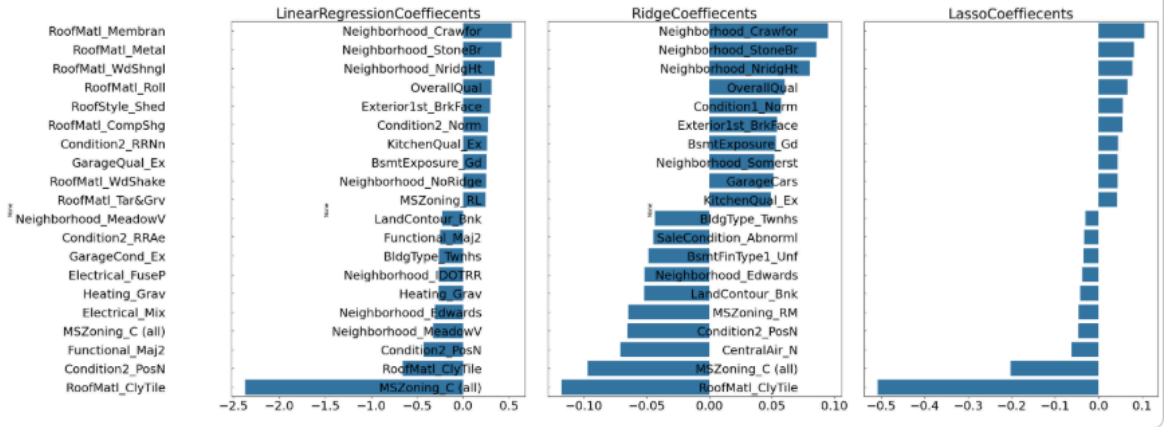
(2)-4. 재학습/예측

```
lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
ridge_reg = Ridge(alpha=12)
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso(alpha=0.001)
lasso_reg.fit(X_train, y_train)

models = [lr_reg, ridge_reg, lasso_reg]
get_rmses(models)

models = [lr_reg, ridge_reg, lasso_reg]
visualize_coefficient(models)
```

```
LinearRegression 로그 변환된 RMSE: 0.132
Ridge 로그 변환된 RMSE: 0.124
Lasso 로그 변환된 RMSE: 0.12
```



- alpha 값 최적화 후 테스트 데이터 세트의 예측 성능이 더 좋아짐
- 모델별 회귀 계수가 많이 달라짐
- 라쏘 모델의 경우, 릿지에 비해 동일한 피처라도 회귀 계수 값이 작음

추가적인 가공

(1) 숫자형 피처의 데이터 분포도 확인

```
from scipy.stats import skew

features_index = house_df.dtypes[house_df.dtypes != 'object'].index
skew_features = house_df[features_index].apply(lambda x : skew(x))

skew_features_top = skew_features[skew_features > 1]
print(skew_features_top.sort_values(ascending=False))
```

```
MiscVal          24.451640
PoolArea         14.813135
LotArea          12.195142
3SsnPorch        10.293752
LowQualFinSF      9.002080
KitchenAbvGr      4.483784
BsmtFinSF2        4.250888
ScreenPorch       4.117977
BsmtHalfBath      4.099186
EnclosedPorch     3.086696
MasVnrArea        2.673661
LotFrontage       2.382499
OpenPorchSF       2.361912
BsmtFinSF1        1.683771
WoodDeckSF        1.539792
TotalBsmtSF       1.522688
MSSubClass        1.406210
1stFlrSF          1.375342
GrLivArea         1.365156
dtype: float64
```

- 원-핫 인코딩이 적용되지 않은 상태여야 함

(2) 최적 alpha 값 찾기

```
house_df[skew_features_top.index] = np.log1p(house_df[skew_features_top.index])
```

```
house_df_ohe = pd.get_dummies(house_df)
y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice', axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target,
                                                    test_size=0.2, random_state=156)

ridge_params = {'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20]}
lasso_params = {'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1, 5, 10]}
print_best_params(ridge_reg, ridge_params)
print_best_params(lasso_reg, lasso_params)
```

```
Ridge 5 CV 시 최적 평균 RMSE 값: 0.1275, 최적 alpha: {'alpha': 10}
Lasso 5 CV 시 최적 평균 RMSE 값: 0.1252, 최적 alpha: {'alpha': 0.001}
```

- 왜곡 정도가 높은 피처를 로그 변환 후 원-핫 인코딩 적용
- 릿지 모델의 경우 최적 alpha값이 12에서 10으로 변경
- 릿지는 최적 평균 RMSE 값이 0.1418에서 0.1275로, 라쏘는 0.14에서 0.1252로 RMSE값이 향상

(3) 전처리

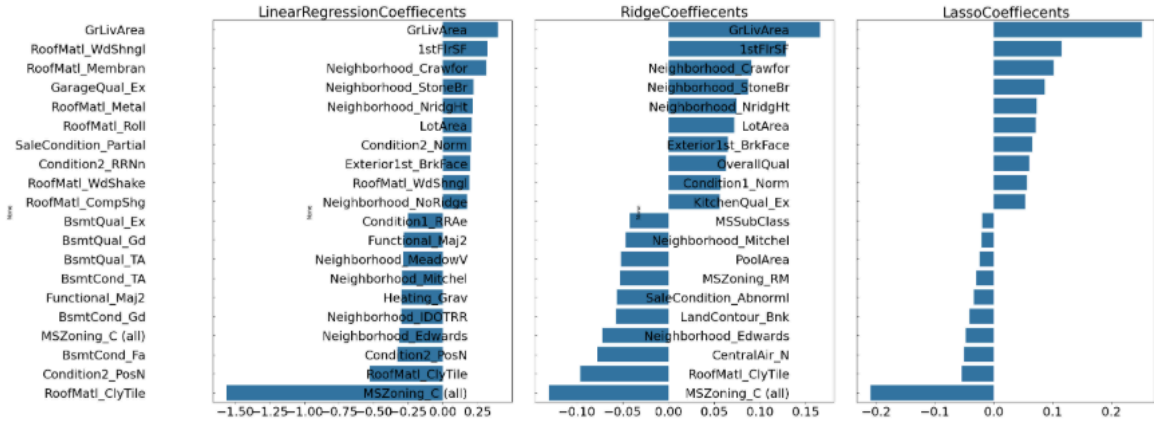
(3)-1. 회귀 계수 시각화

```
lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
ridge_reg = Ridge(alpha=10)
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso(alpha=0.001)
lasso_reg.fit(X_train, y_train)

models = [lr_reg, ridge_reg, lasso_reg]
get_rmses(models)

models = [lr_reg, ridge_reg, lasso_reg]
visualize_coefficient(models)
```

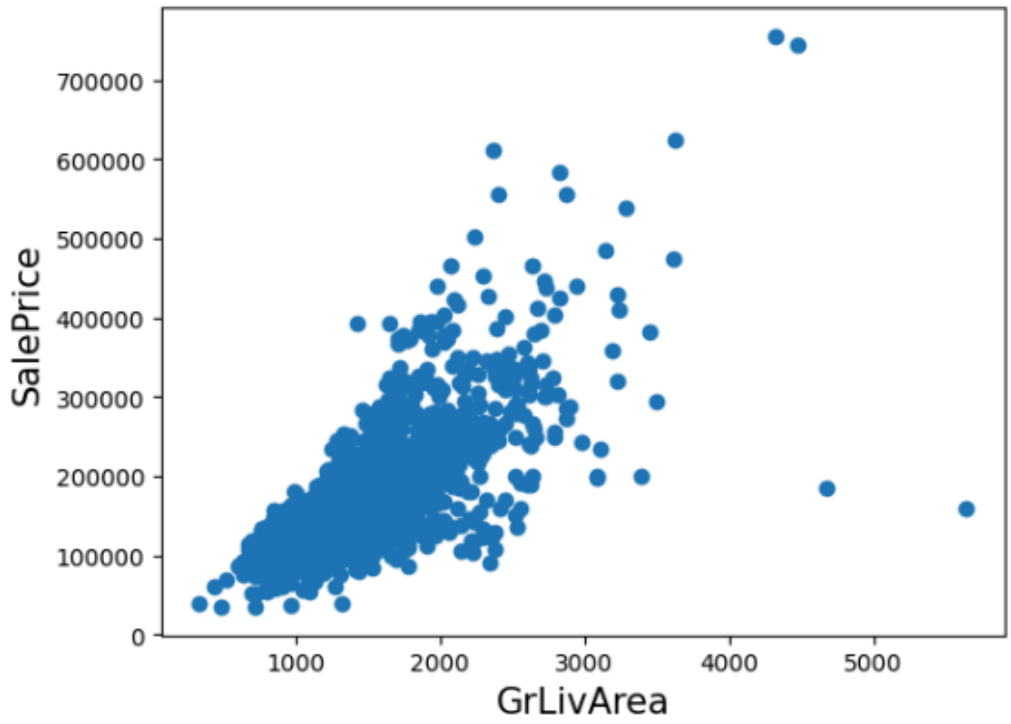
```
LinearRegression 로그 변환된 RMSE: 0.128
Ridge 로그 변환된 RMSE: 0.122
Lasso 로그 변환된 RMSE: 0.119
```



- `GrLivArea` 즉, 주거 공간 크기의 회귀 계수가 가장 높음

(3)-2. 관계 시각화

```
plt.scatter(x=house_df_org['GrLivArea'], y = house_df_org['SalePrice'])
plt.ylabel('SalePrice', fontsize=15)
plt.xlabel('GrLivArea', fontsize=15)
plt.show()
```



- GrLivArea 와 target 값인 SalePrice 의 관계
- 양의 상관도가 매우 높음
- 오른쪽 2개의 데이터는 일반적인 관계에서 너무 어긋나 있음 -> 이상치로 간주

(3)-3. 이상치 처리

```
cond1 = house_df_ohe['GrLivArea'] > np.log1p(4000)
cond2 = house_df_ohe['SalePrice'] < np.log1p(500000)
outlier_index = house_df_ohe[cond1 & cond2].index

print('이상치 레코드 index:', outlier_index.values)
print('이상치 삭제 전 house_df_ohe shape:', house_df_ohe.shape)

house_df_ohe.drop(outlier_index, axis=0, inplace=True)
print('이상치 삭제 후 house_df_ohe shape:', house_df_ohe.shape)
```

```
이상치 레코드 index: [ 523 1298]
이상치 삭제 전 house_df_ohe shape: (1460, 270)
이상치 삭제 후 house_df_ohe shape: (1458, 270)
```

- 2개의 데이터가 삭제 됨

(4) 최적화

(4)-1. 최적화

```
y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice', axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target,
                                                    test_size=0.2, random_state=156)

ridge_params = {'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20]}
lasso_params = {'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1, 5, 10]}
print_best_params(ridge_reg, ridge_params)
print_best_params(lasso_reg, lasso_params)
```

```
Ridge 5 CV 시 최적 평균 RMSE 값: 0.1125, 최적 alpha: {'alpha': 8}
Lasso 5 CV 시 최적 평균 RMSE 값: 0.1122, 최적 alpha: {'alpha': 0.001}
```

- 이상치 제거 후, 예측 수치가 매우 크게 향상됨
- 릿지 모델의 경우 최적 alpha 값이 8로 변함

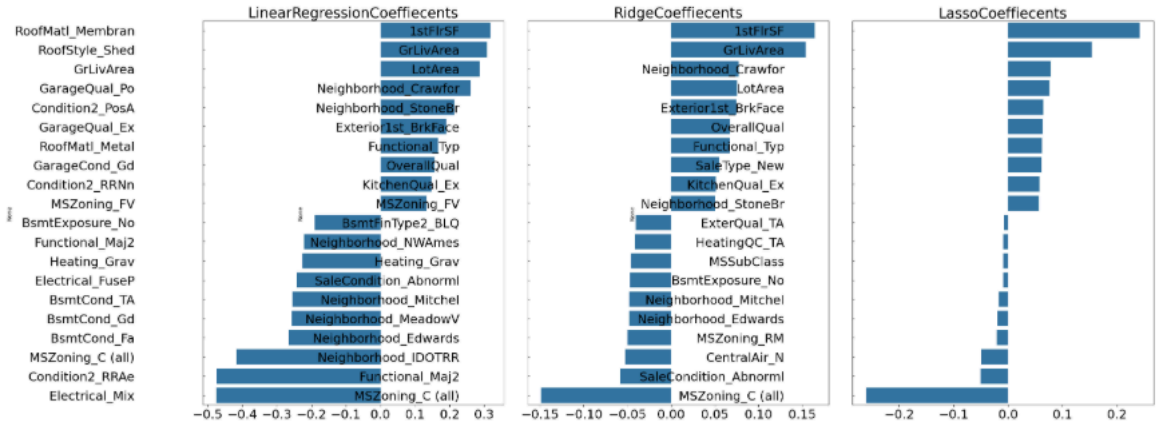
(4)-2. 회귀 계수 시각화

```
lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
ridge_reg = Ridge(alpha=10)
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso(alpha=0.001)
lasso_reg.fit(X_train, y_train)

models = [lr_reg, ridge_reg, lasso_reg]
get_rmses(models)

models = [lr_reg, ridge_reg, lasso_reg]
visualize_coefficient(models)
```

```
LinearRegression 로그 변환된 RMSE: 0.129
Ridge 로그 변환된 RMSE: 0.103
Lasso 로그 변환된 RMSE: 0.1
```



회귀 트리 모델 학습/예측/평가

(1) XGBoost

```
from xgboost import XGBRegressor

xgb_params = {'n_estimators':[1000]}
xgb_reg = XGBRegressor(n_estimators=1000, learning_rate=0.05,
                       colsample_bytree=0.5, subsample=0.8)
print_best_params(xgb_reg, xgb_params)
```

```
XGBRegressor 5 CV 시 최적 평균 RMSE 값: 0.1205, 최적 alpha: {'n_estimators': 1000}
```

- 5 폴드 세트 평균 RMSE가 약 0.1205

(2) LightGBM

```
from lightgbm import LGBMRegressor

lgbm_params = {'n_estimators':[1000]}
lgbm_reg = LGBMRegressor(n_estimators=1000, learning_rate=0.05, num_leaves=4, subsample=0.6,
                         colsample_bytree=0.4, reg_lambda=10, n_jobs=-1, verbose=-1)
print_best_params(lgbm_reg, lgbm_params)
```

```
LGBMRegressor 5 CV 시 최적 평균 RMSE 값: 0.1179, 최적 alpha: {'n_estimators': 1000}
```

- 5 폴드 세트 평균 RMSE가 약 0.1179

회귀 모델의 예측 결과 혼합을 통한 최종 예측

(1) 릿지 모델과 라쏘 모델

```
def get_rmse_pred(preds):
  for key in preds.keys():
    pred_value = preds[key]
    mse = mean_squared_error(y_test, pred_value)
    rmse = np.sqrt(mse)
    print('{} 모델의 RMSE: {}'.format(key, rmse))

ridge_reg = Ridge(alpha=8)
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso(alpha=0.001)
lasso_reg.fit(X_train, y_train)

ridge_pred = ridge_reg.predict(X_test)
lasso_pred = lasso_reg.predict(X_test)

pred = 0.4 * ridge_pred + 0.6 * lasso_pred
preds = {'최종 혼합':pred,
         'Ridge':ridge_pred, 'Lasso':lasso_pred}

get_rmse_pred(preds)
```

```
최종 혼합 모델의 RMSE: 0.10006075517615193
Ridge 모델의 RMSE: 0.10340697165289348
Lasso 모델의 RMSE: 0.10024171179335342
```

- (릿지 모델 예측값 * 0.4) + (라쏘 모델 예측값 * 0.6)
- 최종 혼합 모델의 RMSE가 개별 모델보다 성능 면에서 약간 개선됨

(2) XGBoost와 LightGBM

```
xgb_reg = XGBRegressor(n_estimators=1000, learning_rate=0.05,
                       colsample_bytree=0.5, subsample=0.8)
lgbm_reg = LGBMRegressor(n_estimators=1000, learning_rate=0.05, num_leaves=4, subsample=0.6,
                         colsample_by_tree=0.4, reg_lambda=10, n_jobs=-1, verbose=-1)
xgb_reg.fit(X_train, y_train)
lgbm_reg.fit(X_train, y_train)
xgb_pred = xgb_reg.predict(X_test)
lgbm_pred = lgbm_reg.predict(X_test)

pred = 0.5 * xgb_pred + 0.5 * lgbm_pred
preds = {'최종 혼합':pred,
         'XGBM':xgb_pred, 'LGBM':lgbm_pred}

get_rmse_pred(preds)
```

```
최종 혼합 모델의 RMSE: 0.09938501318547492
XGBM 모델의 RMSE: 0.10309707326102836
LGBM 모델의 RMSE: 0.10362576971641672
```

- (xgbm 예측값 * 0.5) + (lgbm 예측값 * 0.5)
- 최종 혼합 모델의 RMSE가 개별 모델보다 성능 면에서 약간 개선됨

스태킹 앙상블 모델을 통한 회귀 예측

```
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error

def get_stacking_base_datasets(model, X_train_n, y_train_n, X_test_n, n_folds):
  kf = KFold(n_splits=n_folds, shuffle=False)
  train_fold_pred = np.zeros((X_train_n.shape[0], 1))
  test_pred = np.zeros((X_test_n.shape[0], n_folds))
  print(model.__class__.__name__, 'model 시작')

  for folder_counter, (train_index, valid_index) in enumerate(kf.split(X_train_n)):
    print('₩t 폴드 세트:', folder_counter, '시작')
    X_tr = X_train_n[train_index]
    y_tr = y_train_n[train_index]
    X_te = X_train_n[valid_index]

    model.fit(X_tr, y_tr)
    train_fold_pred[valid_index, :]=model.predict(X_te).reshape(-1, 1)
    test_pred[:, folder_counter] = model.predict(X_test_n)

  test_pred_mean = np.mean(test_pred, axis=1).reshape(-1, 1)

  return train_fold_pred, test_pred_mean
```

```
X_train_n = X_train.values
X_test_n = X_test.values
y_train_n = y_train.values

ridge_train, ridge_test = get_stacking_base_datasets(ridge_reg, X_train_n, y_train_n, X_test_n, 5)
lasso_train, lasso_test = get_stacking_base_datasets(lasso_reg, X_train_n, y_train_n, X_test_n, 5)
xgb_train, xgb_test = get_stacking_base_datasets(xgb_reg, X_train_n, y_train_n, X_test_n, 5)
lgbm_train, lgbm_test = get_stacking_base_datasets(lgbm_reg, X_train_n, y_train_n, X_test_n, 5)



Stack_final_X_train = np.concatenate((ridge_train, lasso_train, xgb_train, lgbm_train), axis=1)
Stack_final_X_test = np.concatenate((ridge_test, lasso_test, xgb_test, lgbm_test), axis=1)

meta_model_lasso = Lasso(alpha=0.0005)

meta_model_lasso.fit(Stack_final_X_train, y_train)
final = meta_model_lasso.predict(Stack_final_X_test)
mse = mean_squared_error(y_test, final)
rmse = np.sqrt(mse)
print('스태킹 회귀 모델의 최종 RMSE 값은:', rmse)
```

```
스태킹 회귀 모델의 최종 RMSE 값은: 0.09663907161218126
```

- RMSE가 약 0.0966로 가장 좋은 성능 평가를 보여줌



조윤서

이전 포스트
[Euron] 머신러닝 스터디 6주차

0개의 댓글

댓글을 작성하세요

댓글 작성

Powered by Stellate